

Data Structures in Java

Lecture 6: Introduction to Stacks and Queues

09/26/2016

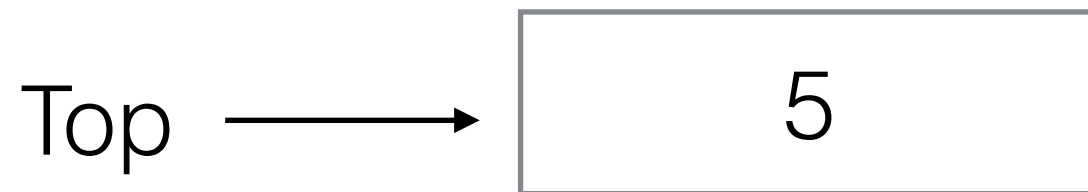
Daniel Bauer

The Stack ADT



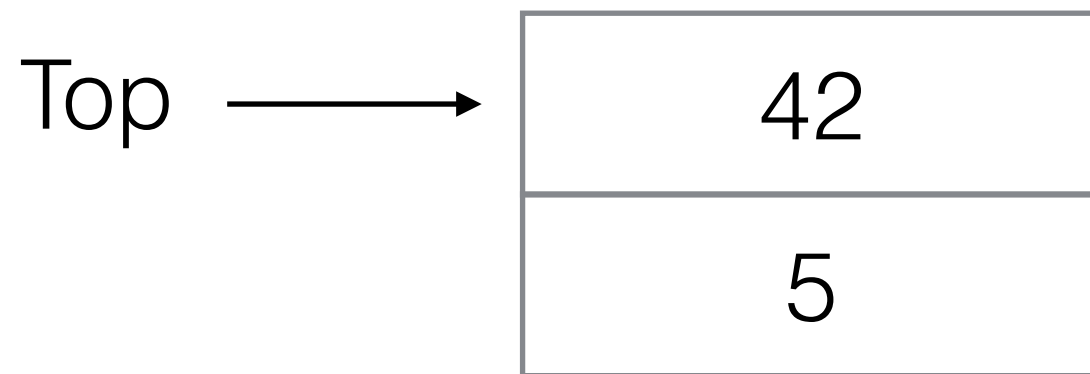
- A Stack S is a sequence of N objects $A_0, A_1, A_2, \dots, A_{N-1}$ with three operations:
 - `void push(x)` - append element x to the end (on “top”) of S .
 - `Object top() / peek()` = returns the last element of S .
 - `Object pop()` - remove and return the last element from S .
- Stacks are also known as **L**ast **I**n **F**irst **O**ut (LIFO) storage.

Stack Example



Stack Example

push(42)



Stack Example

push(42)

push(23)

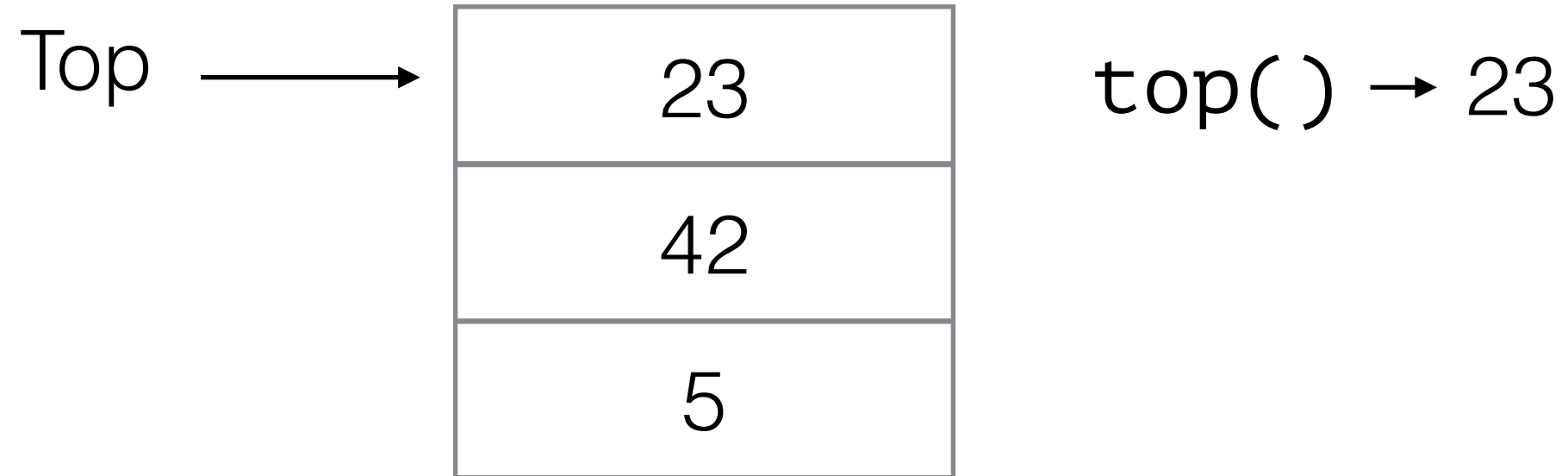
Top →

23
42
5

Stack Example

push(42)

push(23)



Stack Example

push(42)

push(23)

push(3)

Top →



top() → 23

Stack Example

push(42)

push(23)

push(3)

pop() → 3

Top →



top() → 23

Implementing Stacks

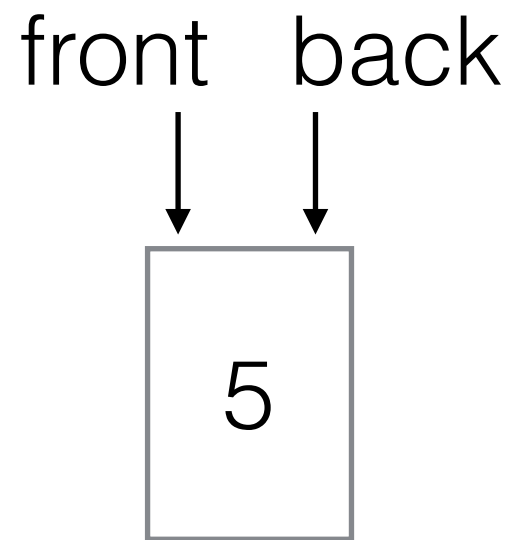
- Think of a Stack as a specialized List:
 - push: Inserts only allowed at the end of the list.
 - pop: Remove only allowed at the end of the list.
- Can implement Stack using any List implementation.
- push and pop run in $O(1)$ time with ArrayList or LinkedList.

The Queue ADT

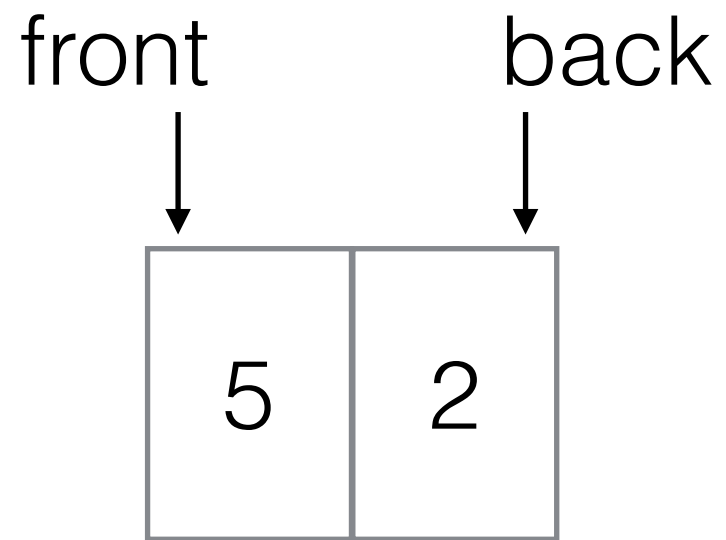


- A Queue Q is a sequence of N objects $A_0, A_1, A_2, \dots, A_{N-1}$
- A_0 is called the front of Q , A_{N-1} is called the back of Q .
- A queue has two operations:
 - `void enqueue(x)` - append element x to the back of Q .
 - `Object dequeue()` - remove and return the front of Q .
- Queues are also known as **F**irst **I**n **F**irst **O**ut (FIFO) storage.

Queue Example

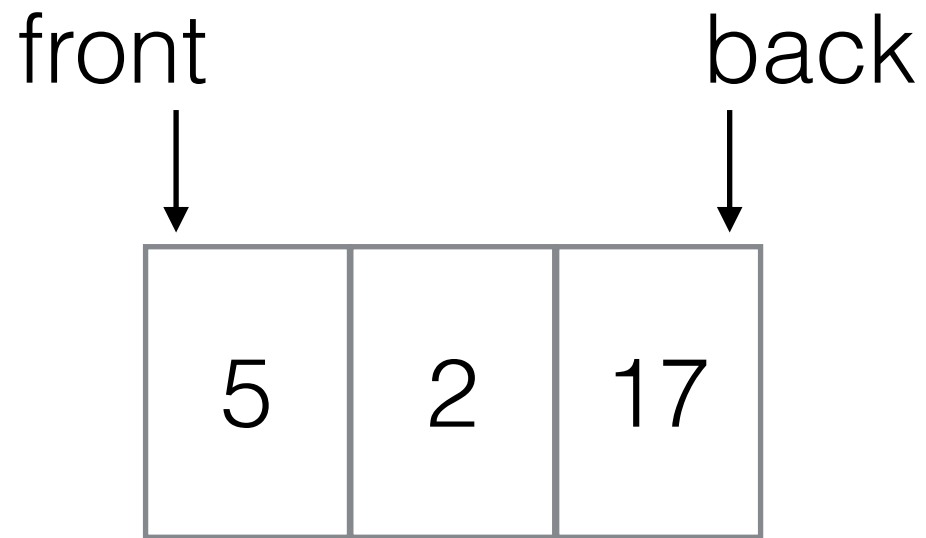


Queue Example



enqueue(2)

Queue Example



enqueue(2) enqueue(17)

Queue Example

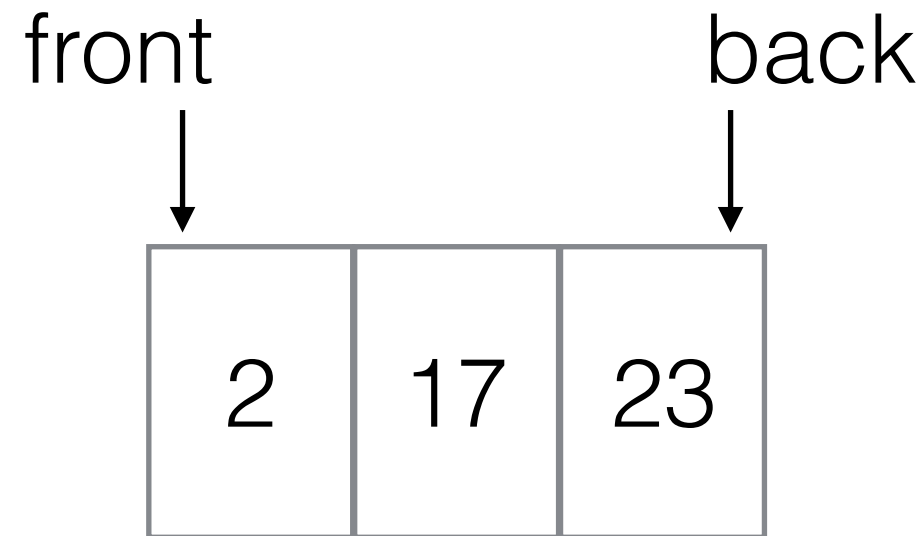


enqueue(2)

enqueue(17)

enqueue(23)

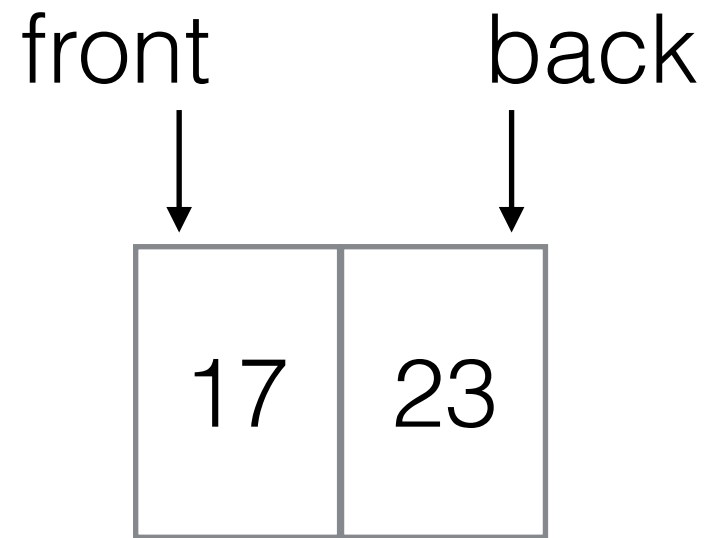
Queue Example



enqueue(2) enqueue(17) enqueue(23)

dequeue() -> 5

Queue Example



enqueue(2) enqueue(17) enqueue(23)

dequeue() -> 5 dequeue() -> 2

A Stack Interface

```
interface Stack<T> {  
    /* Push a new item x on top of the stack */  
    public void push(T x);  
    /* Remove and return the top item of the stack */  
    public T pop();  
    /* Return the top item of the stack without removing it */  
    public T top();  
}
```

Using a List to implement a Stack

```
public class LinkedListStack<T> extends MyLinkedList<T>
                                   implements Stack<T> {

    public void push(T x) {
        add(size(), x);
    }

    public T pop() {
        return remove(size()-1);
    }

    public T top() {
        return get(size()-1);
    }
}
```

Implementing a Stack using an array

(see code)

A Queue Interface

```
interface Queue<T> {  
    /**  
     * Insert a new item at the back of the queue  
     */  
    public void enqueue(T x);  
    /**  
     * Remove and return the next item from the  
     * front of the queue.  
     */  
    public T dequeue();  
    /**  
     * Return the next item from the  
     * front of the queue but do not remove it.  
     */  
    public T getFront();  
}
```

The Java Collection API

```
package java.util;

interface Collection<E> extends Iterable<E> {
    boolean add(E e);
    boolean addAll(Collection<? extends E> c);
    void clear();
    boolean contains(Object o);
    boolean containsAll(Collection<?> c);
    boolean isEmpty();
    Iterator<E> iterator(); // via Iterable
    boolean remove(Object o);
    boolean removeAll(Collection<?> c);
    boolean retainAll(Collection<?> c);
    int size();
    Object[] toArray();
    <T> T[] toArray(T[] a);
}
```

Java API List Interface

```
package java.util;

interface List<E> extends Collection<E> {
    E get(int index);
    int indexOf(Object o);
    int lastIndexOf(Object o);
    E remove(int index);
    E set(int index, E element);
    List<E> subList(int fromIndex, int toIndex)
}
```

Java Queue Interface

```
package java.util;

interface Queue<E> extends Collection<E> {
    /* These methods throw exception on failure */
    boolean add(E e); // enqueue
    E remove(); // dequeue
    E element(); // Retrieve, but do not remove, front
    /* These methods return null on failure */
    boolean offer(E e); //enqueue
    E poll(); // dequeue
    E peek();
}
```


Java Deque Interface

A linear collection that supports element insertion and removal at both ends. The name deque is short for "double ended queue" and is usually pronounced "deck"

```
package java.util;

interface Deque<E> extends Collection<E> {
    /* These methods throw exception on failure */
    boolean addFirst(E e);
    boolean addLast(E e);
    E removeFirst(); // dequeue
    E removeLast();  // dequeue
    E getFirst();
    E getLast();
    /* These methods return null on failure */
    ...
}
```


Dequeues can be Queues or Stacks

- **Stack view:**

addFirst(E e)	~ push(E e)
E removeFirst()	~ E pop()
E getFirst()	~ E peek() / top()

- **Queue view:**

addLast(E e)	~ enqueue(E e) / add(E e)
E removeFirst()	~ dequeue() / remove()
E getFirst()	~ element()

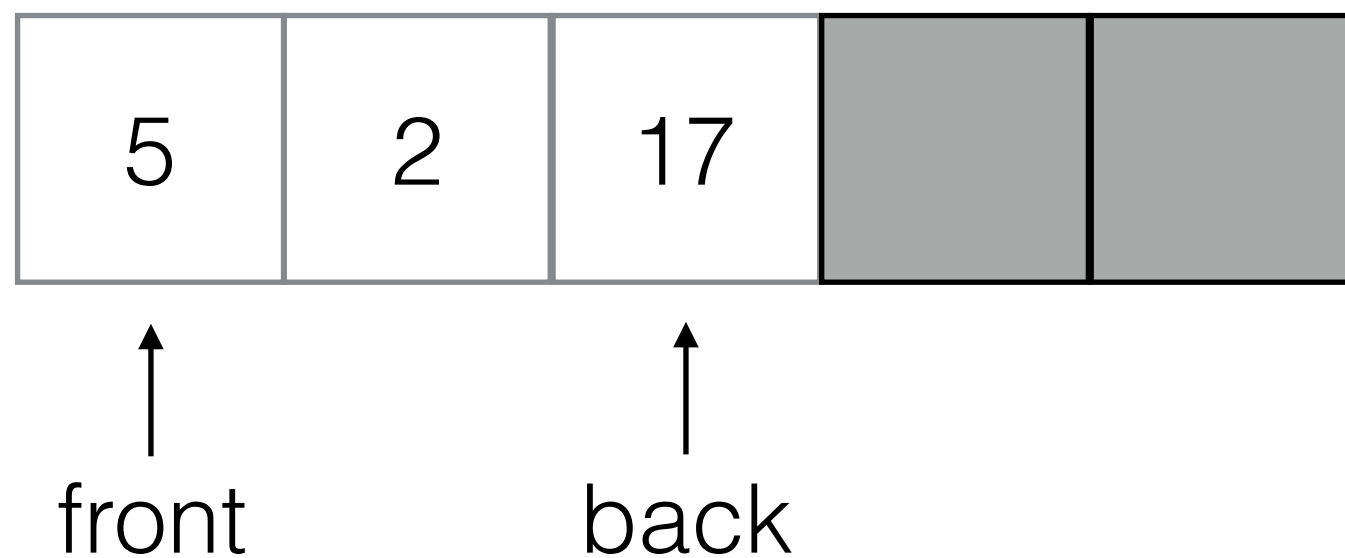
Implementing Queues

- Think of a Queue as a specialized List:
 - **enqueue**: Inserts only allowed at the end of the list.
 - **dequeue**: Remove only allowed at the beginning of the list.
- Can implement Queue using LinkedList implementation or using arrays.

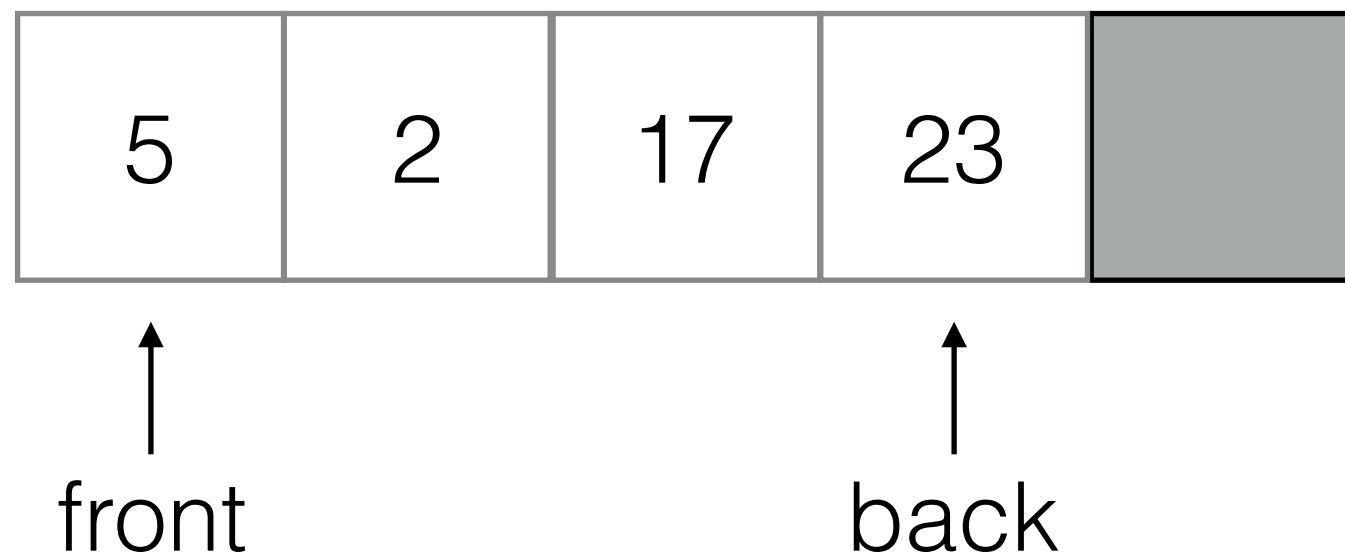
Implementing Queues

- Think of a Queue as a specialized List:
 - **enqueue**: Inserts only allowed at the end of the list.
 - **dequeue**: Remove only allowed at the beginning of the list.
- Can implement Queue using LinkedList implementation or using arrays.
 - enqueue and dequeue run in $O(1)$ time with LinkedList.
 - What happens during dequeue in an Array?

Deque on ArrayLists

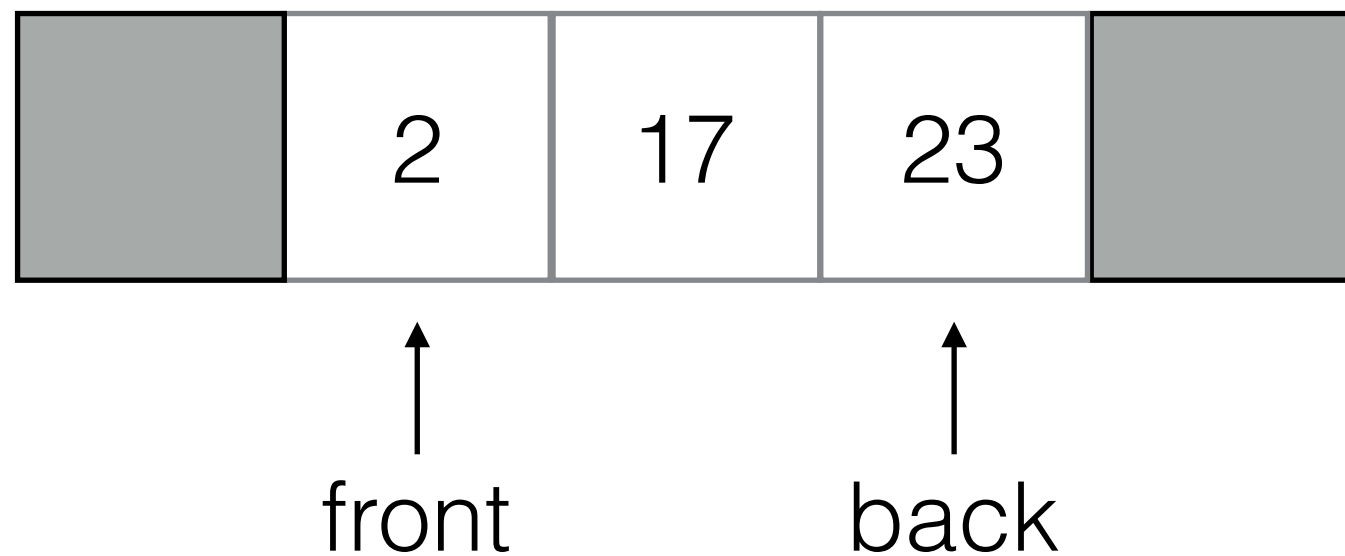


Dequeue on ArrayLists



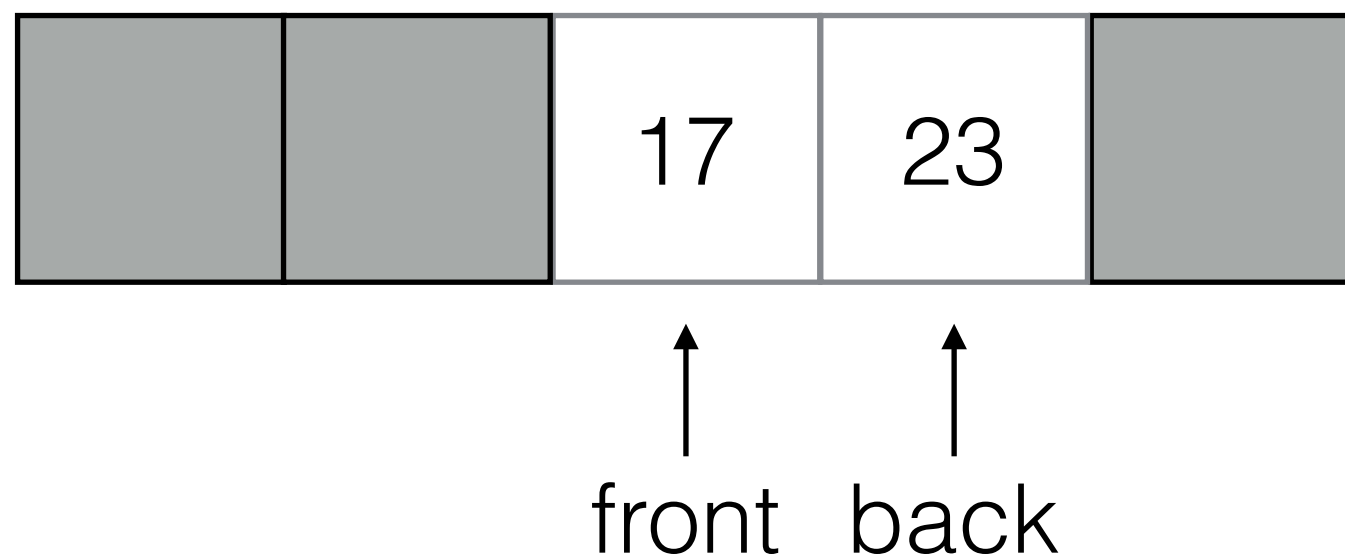
enqueue(23)

Deque on ArrayLists



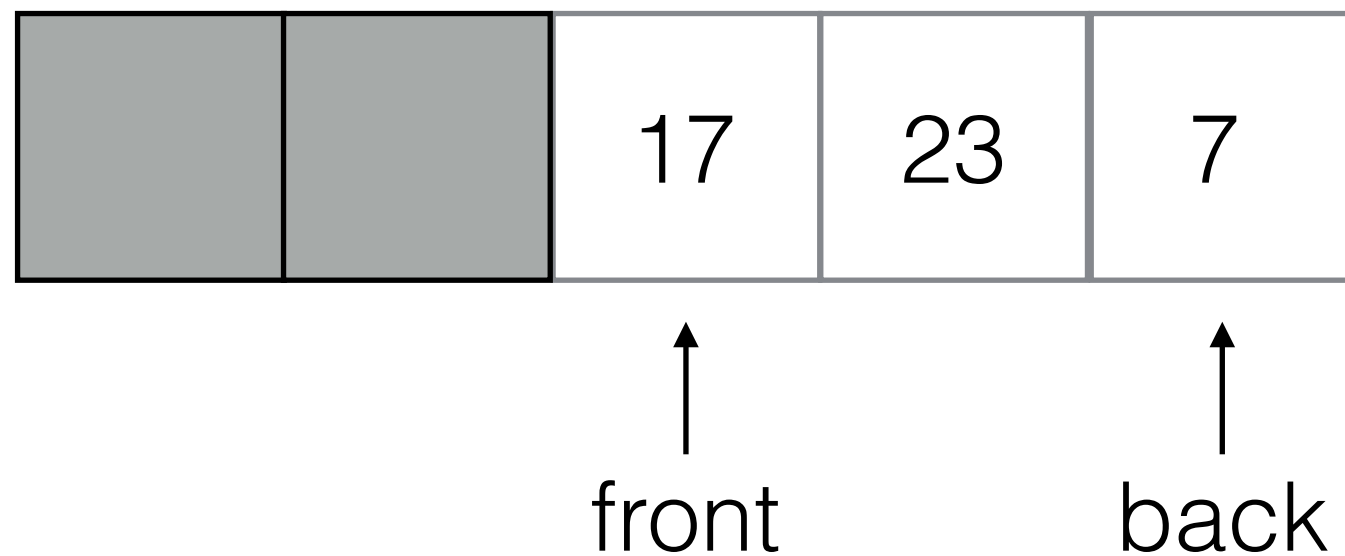
enqueue(23) dequeue() -> 5

Deque on ArrayLists



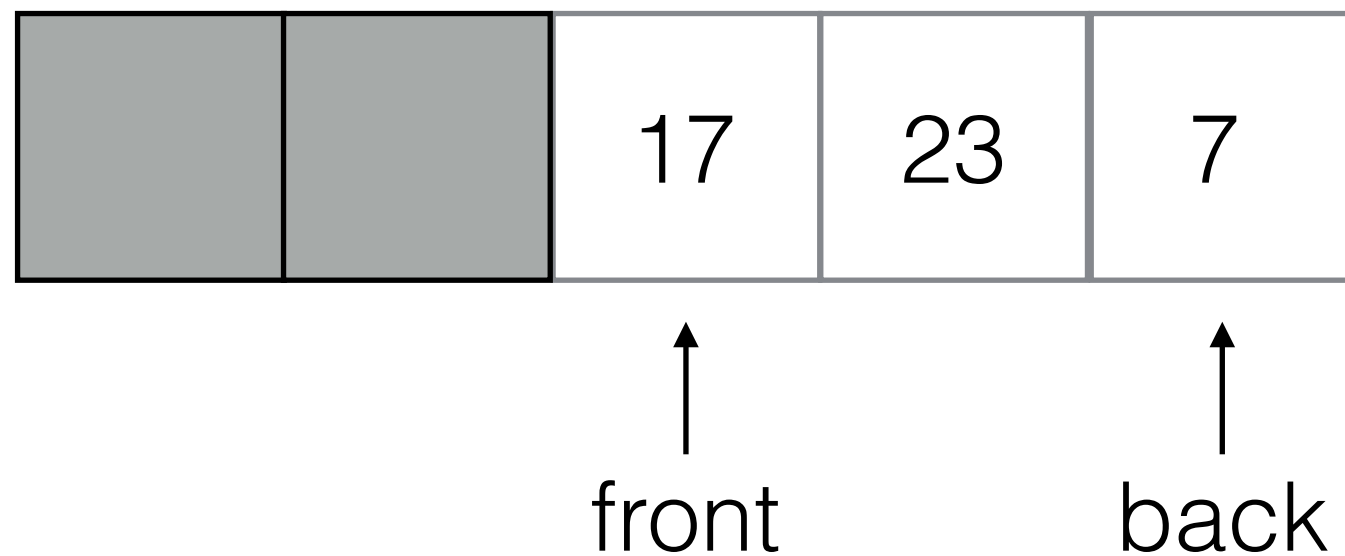
enqueue(23) dequeue() -> 5 dequeue() -> 2

Deque on ArrayLists



enqueue(23) dequeue() -> 5 dequeue() -> 2
enqueue(7)

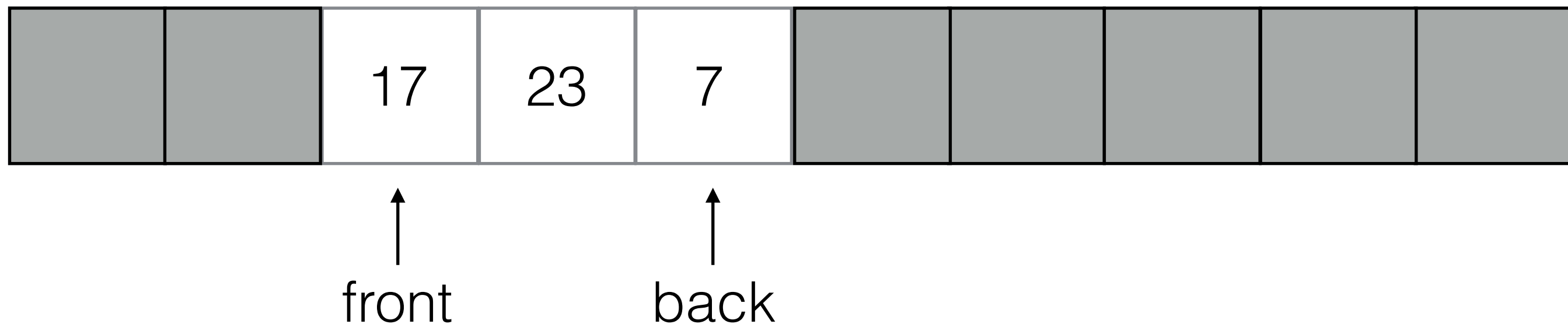
Deque on ArrayLists



enqueue(23) dequeue() -> 5 dequeue() -> 2
enqueue(7) enqueue(42)

Deque on ArrayLists

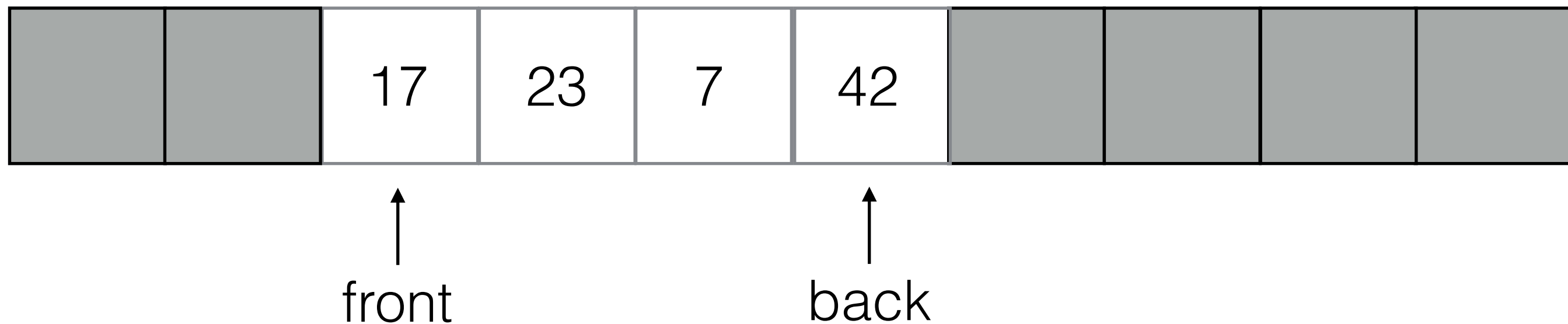
Need to reserve larger array, even though there is plenty of space at the beginning of the array.



enqueue(42)

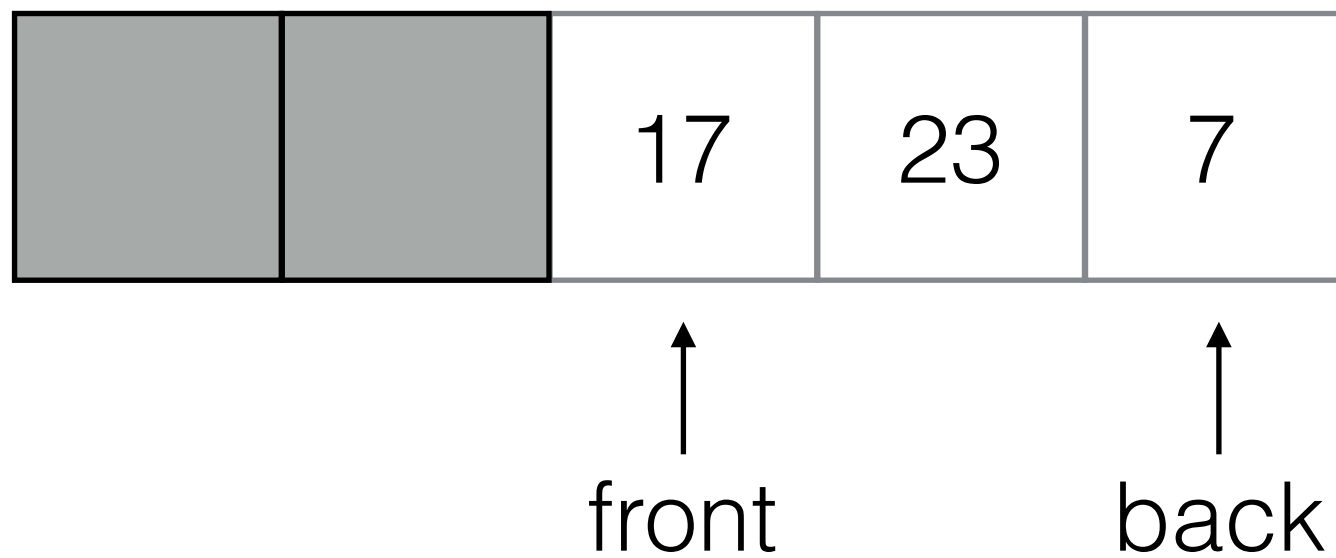
Deque on ArrayLists

Need to reserve larger array, even though there is plenty of space at the beginning of the array.



enqueue(42)

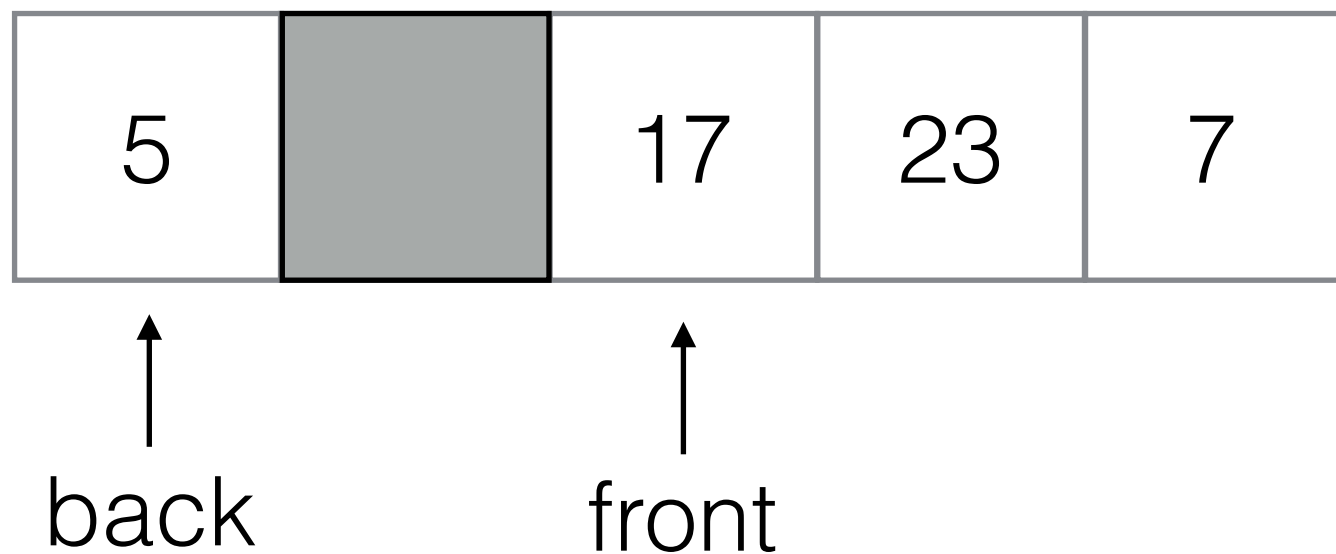
Circular Array



enqueue(42)

enqueue(9)

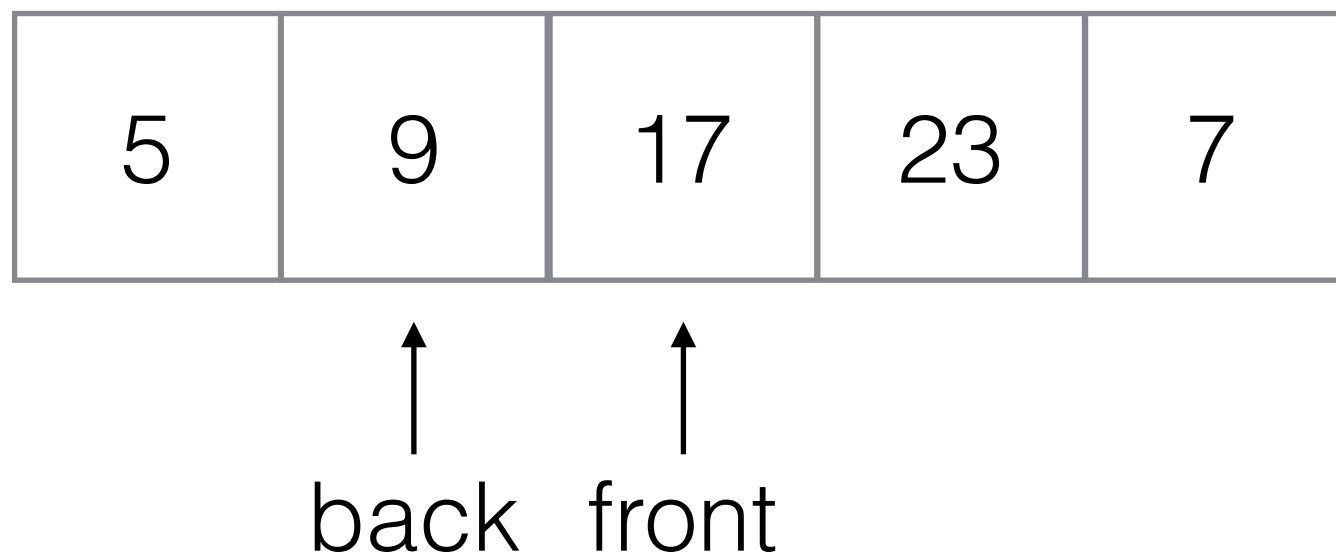
Circular Array



enqueue(42)

enqueue(9)

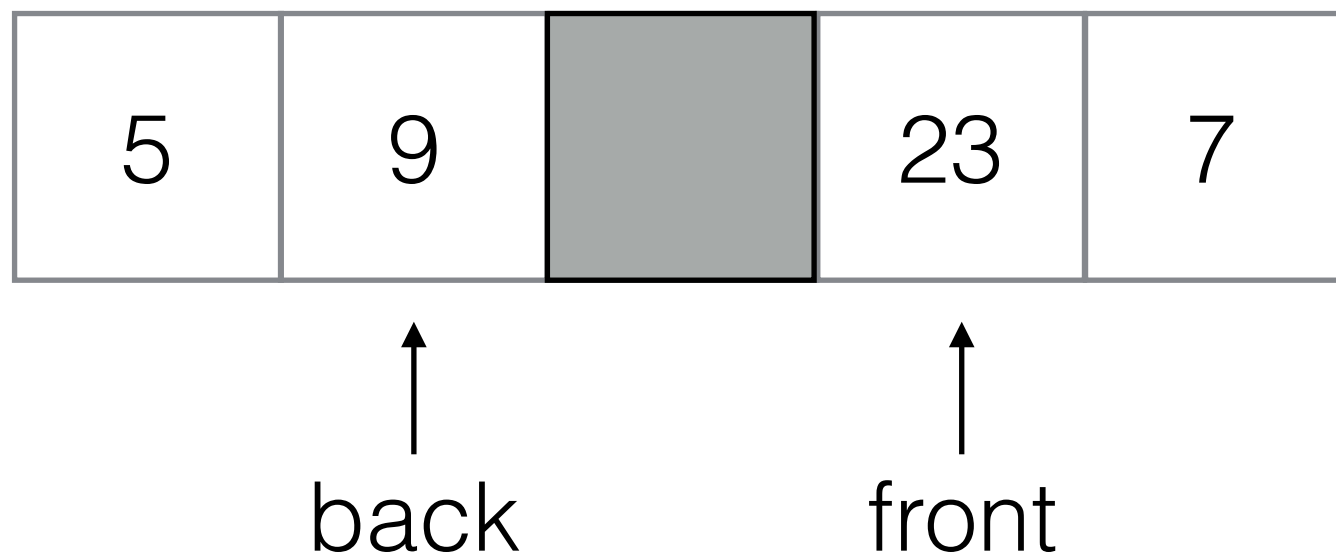
Circular Array



enqueue(42)

enqueue(9)

Circular Array



enqueue(42)

enqueue(9)

dequeue() -> 17

Circular Array Implementation

(see code)